ESOFT METRO CAMPUS • FACULTY OF COMPUTING • BIT (HONS) DEGREE

Interim Viva Voce Defense Guide: Top 12 Questions & Answers

Dutch Point Negombo Web-Based Hotel Management System • Technical Rationale, Code Evidence & Examiner Q&A;
Team Lead: *I.D.P.N Iddamalgoda (E269559)* • ***Team Members:*** *H.M.Y.R Herath (E267614), Y.M.A.Y Bandara (E267609), A.G.H.H Jinadasa (E267612)*

---

1. Core Business Logic & Concurrency Control

Q1: How did you prevent double-booking in your system?

Why the examiner asks this: *Checks whether you implemented real backend validation or just relied on frontend date pickers.*

Winning Model Answer:
"Sir/Madam, we prevent double booking at the database query tier in BookingsController.cs using Allen's Interval Algebra. Before saving a booking, the backend queries whether any existing confirmed booking satisfies both conditions: ExistingCheckIn < RequestedCheckOut ***AND*** *ExistingCheckOut > RequestedCheckIn. If both evaluate to true, an overlap exists. The system halts the transaction, writes zero records to MySQL, and returns an* ***HTTP 409 Conflict*** *error carrying diagnostic conflict details to the frontend."*

Q2: What happens if two users click 'Confirm Booking' at the exact same second for the same room?

Why the examiner asks this: *Tests your understanding of distributed race conditions and database transaction concurrency.*

Winning Model Answer:
"Even if both users see the room as available and open the modal simultaneously, MySQL executes the transactions sequentially under ACID transaction isolation. The first transaction commits successfully. When the second transaction executes, its date-overlap check queries the updated database state, detects the collision, and rejects the second user with an HTTP 409 Conflict without corrupting room inventory."

Q3: Why didn't you just change the Room table status to 'Occupied' when a room is booked?

Why the examiner asks this: *A classic examiner trap! Amateur systems mutate a static status column, causing permanent room lockouts.*

Winning Model Answer:
"Because room availability is ***temporal****, not permanent. A room occupied today will be completely vacant next week. If we change the room table's status to 'Occupied', the room becomes permanently unavailable for all future dates. Instead, our system uses* ***dynamic in-memory projection****: we query only the bookings overlapping with the user's specific requested dates, leaving the static room record intact."*

2. System Architecture & Component Integration

Q4: How does your React frontend communicate with your ASP.NET Core backend?

Why the examiner asks this: *Verifies if you understand client-server networking, port allocation, and REST API standards.*

Winning Model Answer:

"They communicate over HTTP using asynchronous RESTful JSON endpoints. The React frontend runs on Vite on port 5173, and the ASP.NET Core backend listens on port 5000. When a user interacts with the UI, React uses fetch() to send requests to endpoints like http://localhost:5000/api/rooms. We configured ***CORS middleware (app.UseCors('AllowAll'))*** *in Program.cs to permit cross-origin communication between the two ports."*

Q5: Why did you choose a Decoupled 3-Tier Architecture instead of a traditional monolithic MVC?

Why the examiner asks this: *Tests your ability to provide academic and technical justification for your architectural choices.*

Winning Model Answer:

"A decoupled architecture separates concerns cleanly: React 19 handles the presentation state via its Virtual DOM without full page reloads, ASP.NET Core encapsulates business validation, and MySQL 8.0 ensures ACID transactional persistence. This allows independent scalability, easier testing via Swagger, and the flexibility to connect future mobile apps to the same API."

3. Security & Role-Based Access Control (RBAC)

Q6: How do you prevent a regular customer from registering as an Administrator?

Why the examiner asks this: *Checks for privilege escalation vulnerabilities in your user onboarding pipeline.*

Winning Model Answer:

"We enforce a server-side anti-privilege escalation guard in AuthController.cs. In the registration endpoint, the backend strictly hardcodes newUser.Role = 'Customer', completely ignoring any role submitted by the client payload. Administrative accounts cannot be self-registered online; they can only be pre-seeded directly in the database."

Q7: Why are passwords stored as plain text in this build, and how will you fix it?

Why the examiner asks this: *They will notice the plain-text password in AuthController.cs and test your awareness of security standards.*

Winning Model Answer:

"Sir/Madam, plain-text credentials were maintained temporarily during the 55% interim stage solely to facilitate rapid examiner testing and evaluation. As documented in our Chapter 06 Future Roadmap, Phase 2 will replace this with salted ***BCrypt.Net (work factor 12)*** *hashing and signed* ***JWT (JSON Web Tokens)*** *to comply with Sri Lanka's Personal Data Protection Act No. 9 of 2022."*

4. Database Design & Relational Data Integrity

Q8: What happens if an admin tries to delete a room that already has active guest reservations?

Why the examiner asks this: *Tests whether your system enforces relational referential integrity or allows orphaned records.*

Winning Model Answer:

"The system strictly prevents it. In RoomsController.cs, before executing a deletion, the backend checks .AnyAsync(b => b.RoomId == id && b.BookingStatus == 'Confirmed'). If active bookings exist, the backend aborts with an HTTP 400 Bad Request error. Furthermore, our MySQL schema enforces foreign key rules (ON DELETE RESTRICT) to prevent orphaned financial records."

Q9: Why did you use DECIMAL(10,2) instead of FLOAT or DOUBLE for room prices in LKR?

Why the examiner asks this: *Tests your knowledge of financial computing and floating-point arithmetic precision.*

Winning Model Answer:
"In financial systems, binary floating-point types (float and double) suffer from IEEE 754 precision drift and rounding errors. We used DECIMAL(10,2) in MySQL and decimal in C# because they represent exact fixed-point values, ensuring accurate billing in Sri Lankan Rupees (LKR)."

5. Quality Assurance, Reflections & Phase 2 Roadmap

Q10: How did you test your backend APIs before connecting them to React?

Why the examiner asks this: *Verifies whether your development followed a disciplined Software Engineering QA methodology.*

Winning Model Answer:
*"We integrated **Swagger UI and OpenAPI** directly at http://localhost:5000/swagger, alongside **Postman**. This allowed us to test all CRUD endpoints, model validations, and simulate double-booking boundary cases independently before integrating the frontend."*

Q11: What was your biggest technical challenge during development?

Why the examiner asks this: *Tests self-reflection, problem-solving maturity, and authentic developer experience.*

Winning Model Answer:
"Our main hurdle was adapting to ASP.NET Core's asynchronous threading pipeline and resolving circular entity loops in Entity Framework Core when serializing related Customer and Booking data. We solved this by designing clean, flat Data Transfer Objects (DTOs). We also resolved a date timezone offset bug where Sri Lanka's UTC+5:30 offset shifted dates backward by one day, which we fixed by enforcing DateTime.UtcNow.Date normalization."

Q12: What is remaining for your final submission (Phase 2)?

Why the examiner asks this: *Checks if you have a realistic, well-defined plan to reach 100% completion.*

Winning Model Answer:
"For Phase 2, we will deliver four key production modules: 1. Salted BCrypt password hashing and JWT authentication; 2. Two-Factor Authentication (TOTP via Google Authenticator) for administrators; 3. An executive analytics dashboard with interactive room schedule timelines; 4. Direct online payment gateway integration (PayHere / Stripe) for live LKR card settlements."